

Aura AI Project Notes

Project architecture, current features, limitations, and next development plan

1. Project Purpose

Aura is a browser-based agentic assistant designed to combine natural-language chat, voice commands, spoken responses, webcam hand gestures, and future third-party app actions in one workspace.

The current application is a functional prototype. Gemini supplies language-model answers, the browser supplies speech recognition and speech synthesis, and MediaPipe supplies local hand landmark detection.

2. Current Technology Stack

Languages

- TypeScript: React logic, browser APIs, Gemini proxy configuration, and type definitions.
- TSX: React UI markup in `src/App.tsx`.
- CSS: Responsive metallic interface styling in `src/App.css` and `src/index.css`.
- HTML: Application shell and favicon metadata in `index.html`.
- JSON: npm configuration and TypeScript/Vite configuration.

Main programs and frameworks

- React 19: Component-based user interface.
- Vite 8: Development server, bundler, and custom development API middleware.
- TypeScript 6: Static type checking and project compilation.
- Node.js and npm: Dependency management and development commands.
- OXlint: Available project lint command.
- MediaPipe Tasks Vision: Browser-side hand landmark detection.
- Google GenAI SDK: Server-side Gemini request handling.

Important dependencies

- `react` and `react-dom`: UI rendering.
- `@google/genai`: Gemini API client.
- `@mediapipe/tasks-vision`: Hand landmark model runtime.
- `vite` and `@vitejs/plugin-react`: Dev server and React build.
- `typescript`: Type checking.
- `oxlint`: Linting.
- `pdfkit`: Local generation of this PDF document.

3. How the Application Works

Frontend entry point

`src/main.tsx` mounts the React application into the `#root` element.

`src/App.tsx` contains the primary assistant workspace, state, browser integrations, voice flow, webcam flow, gesture logic, OAuth popup UI, and chat rendering.

Chat and Gemini flow

1. The user submits typed text or a voice transcript.
2. Aura adds the user message to the local conversation history.
3. The frontend sends recent messages and the selected language to `POST /api/chat`.
4. The Vite middleware in `vite.config.ts` reads `GEMINI\_API\_KEY` from the server environment.
5. The server calls Gemini using `gemini-3.6-flash`.
6. The server returns the answer to React.
7. React renders the answer and sends it to browser speech synthesis.

The API key is intentionally kept server-side and is not placed in React source code.

Voice command flow

1. The user selects a language and optional installed system voice.
2. The microphone is preflighted through `navigator.mediaDevices.getUserMedia``.
3. Browser Speech Recognition captures the transcript.
4. The transcript is sent to Gemini.
5. Aura selects a matching browser voice where available.
6. Speech Synthesis reads the answer aloud.

Supported language choices currently include English, English UK, Spanish, French, German, Hindi, Urdu, Japanese, and Portuguese. Actual spoken output depends on voices installed by the operating system and browser.

Camera and gesture flow

1. The user enables the webcam through `getUserMedia``.
2. The video is rendered in the Vision Sensor panel.
3. MediaPipe loads the hand landmark model and WASM runtime from public CDNs.
4. Each video frame is analyzed locally in the browser.
5. Hand landmarks and a rounded tracking outline are drawn on a canvas over the video.
6. Landmark geometry is classified into gesture names and actions.

Implemented gesture mappings include pinch in, pinch and spread, peace, pointing index, thumbs up, thumbs down, rock horns, three-finger claw, open palm, horizontal swipes, and vertical swipe up.

4. Current User Interface

- Animated metallic black, graphite, grey, white, and cyan theme.
- Aura GIF in the brand and assistant message avatars.
- Transparent neon Aura favicon.
- Full-screen Welcome to Aura overlay on page load.
- Time-aware greeting: Good morning, Good afternoon, Good evening, or Good night.
- Conversation panel with text input, voice button, language selector, voice selector, and suggestion prompts.
- Vision Sensor panel with webcam preview, gesture outline, motion status, captured gesture, and action status.
- OAuth connections dialog for Calendar, Gmail, Spotify, Microsoft 365, and WhatsApp Business.

5. Configuration and Commands

From `D:\AuraAI\aura-ai``:

```
npm install
npm run dev
npm run build
npm run lint
npm run preview
```

Environment values are stored in `.env``, which is ignored by Git. Start from `.env.example``.

Required for Gemini:

GEMINI_API_KEY=your_gemini_key

OAuth public client ID placeholders are also listed in `.env.example``. Provider client secrets and access tokens must remain server-side.

6. Known Limitations

- Gemini free-tier quota can return HTTP 429 after the project quota is exhausted. More API keys in the same project do not increase quota.
- Browser speech recognition is most reliable in Chrome and Edge.
- Urdu and other non-English speech output requires a matching operating-system speech voice. Gemini text generation and browser speech output are separate capabilities.
- MediaPipe requires network access to download its model and WASM runtime unless those assets are self-hosted.
- Gesture classification is heuristic and should be stabilized with confidence thresholds, cooldowns, and calibration before production use.
- OAuth popup URLs open provider consent screens, but secure callback handling and provider token storage still need to be implemented.
- Calendar, Gmail, Spotify, Microsoft 365, and WhatsApp actions are not fully connected until provider OAuth credentials and server-side integrations are configured.

7. Next Development Plan

Phase 1: Stabilize the prototype

- Self-host the MediaPipe model and WASM assets.
- Add explicit camera and microphone diagnostics.
- Add gesture confidence scores, debounce timers, and configurable sensitivity.
- Add an offline/local response mode for quota errors.
- Add automated tests for gesture classification and chat request handling.

Phase 2: Complete voice experience

- Add push-to-talk and optional wake-word support.
- Add streaming Gemini responses for lower perceived latency.
- Add a speech queue so responses do not overlap.
- Add installed-voice detection and clear language fallback controls.
- Add conversation cancellation while Gemini is responding.

Phase 3: Secure integrations

- Implement `/oauth/callback` on the server.
- Exchange authorization codes server-side using PKCE where supported.
- Encrypt refresh tokens and store them in a database.
- Add Google Calendar and Gmail actions first.
- Add Spotify playback controls.
- Add Microsoft 365 support.
- Treat WhatsApp as a separate Meta Business webhook and server integration.

Phase 4: Agent actions and production readiness

- Add explicit tool/function schemas for reminders, calendar events, music, mail, and browser actions.
- Ask for confirmation before destructive or externally visible actions.
- Add authentication, rate limits, audit logs, and privacy controls.
- Add deployment configuration and HTTPS.
- Add observability for model latency, quota, camera failures, and speech failures.

8. Current Validation

The production build currently passes with:

```
npm run build
```

The app remains a development prototype until OAuth callbacks, secure token storage, self-hosted vision assets, automated tests, and production deployment are completed.